

CASE STUDY

Neutria AI

AI-powered nutrition coaching backend, built independently by Abi (Agentic AI Engineer / Python Backend Engineer)

Repository analyzed: github.com/abizaid24/Neutria-AI-Backend (backend service — Python/FastAPI/MongoDB/Gemini/MCP)

A. Full Case Study

01 — Overview

Project	Neutria AI — AI-Powered Nutrition Coach Backend
Type	Production-shaped reference backend, independent portfolio project
Role	Sole engineer — architecture, backend, AI integration, MCP server
Core stack	Python, FastAPI, MongoDB (Motor), Google Gemini (Vision + Chat), JWT, Model Context Protocol
Purpose	Let a user photograph a meal and get a reliable calorie/macro breakdown, without the app blindly trusting a single AI call for the numbers that matter

Neutria AI is a backend system for an AI-assisted nutrition-tracking product: a user uploads a meal photo, Gemini Vision identifies the food items and estimates portions, and the system resolves calories and macros through a deterministic, multi-source lookup rather than asking the AI to invent nutrition numbers. The same backend also powers a conversational nutrition coach with memory of the user's own meal history, and exposes its core capabilities as MCP tools so autonomous AI clients (like Claude Desktop) can log meals or check progress directly.

02 — The Problem

Before writing any code, the starting point was a simple observation from looking around the fitness/nutrition app space: there is no shortage of calorie trackers and AI-powered food scanners. The gap that stood out was narrower and more specific than "build another tracker" — it was about **trust in the numbers**. Two things stood out in particular:

- **Cost/accessibility** — many existing AI-powered food-scanning tools sit behind subscription paywalls for what is, underneath, a single vision-model call.
- **Reliability of the analysis itself** — the entire value of a food-scanning app collapses if the calorie and macro numbers it hands back are just whatever an AI model guessed from a photo, with no way to sanity-check that guess.

Manual food logging is tedious; that part is well understood. The harder, less-discussed problem is that once you *do* automate it with AI, you've traded tedium for a new failure mode: numbers that look precise but are actually a single ungrounded inference.

(These are observations from the author's own research into existing apps, not benchmarked claims about specific competitors.)

03 — Research & Discovery

Research was done by browsing the Google Play Store and other app marketplaces, evaluating existing fitness, wellness, and AI-powered food-tracking apps directly rather than working from secondary sources. This surfaced the

two recurring themes above — pricing friction and analysis reliability — and shaped a concrete question to build against:

Can I build an AI-powered nutrition application that makes food logging easier while placing strong emphasis on the reliability of nutritional analysis?

That question is the actual design driver behind the backend's most distinctive architectural choice, covered in Section 08.

04 — Product Vision

Neutria AI's premise is that **AI and deterministic computation should do different jobs**. Gemini Vision is well suited to a perceptual task it's actually good at — "what food is this, and roughly how much of it is there." It is not asked to also be the source of truth for calories and macros, which is a lookup/arithmetic problem, not a perception problem. The product philosophy that follows: let AI *identify*, and let structured data and deterministic math *quantify*.

On top of that, the coaching layer is meant to feel personal rather than generic — the AI coach is given the user's actual profile, goal, and recent meals as context on every turn, and is instructed to match the user's own language and style (English, Urdu, Hindi, or a natural Hinglish/Roman-Urdu mix) rather than defaulting to English.

05 — Solution

The system's data flow, as implemented:

```
User photographs a meal
↓
Image validated (MIME whitelist, size limit, Pillow integrity check)
↓
Gemini Vision identifies foods + portion estimates (JSON)
↓
Each food resolved to calories/macros via 3-tier nutrition lookup
↓
Totals aggregated → AI coach writes a short, personalized note
↓
Meal persisted in MongoDB → daily progress snapshot recalculated
↓
User sees results; can chat with the coach using real history as context
```

Only the "identify" step is a single AI call with no independent check; every numeric value downstream of it goes through the deterministic lookup path described next.

06 — Technical Architecture

The backend enforces a **strict three-layer separation**, applied consistently across every domain (auth, users, meals, AI, nutrition, progress):

```
Router (HTTP) → Service (business logic) → Repository (raw Motor/MongoDB queries)
```

Routers never touch MongoDB directly, and services never touch HTTP request/response objects — confirmed directly in code (e.g. `meal_router.py` only ever calls `meal_service`, which in turn calls `meal_repository` and other services; it never imports `Motor`). This keeps HTTP concerns, business rules, and data access independently testable and swappable.

Other architectural decisions evidenced in the repository:

- **FastAPI + Pydantic v2** for request/response validation, with a single `AppBaseModel` shared across all domain

models.

- **MongoDB via Motor** (the official async driver), with a full CRUD surface split cleanly per domain: `/auth` , `/users` , `/meals` , `/ai` , `/nutrition` , `/progress` , `/history` , all mounted under `/api/v1` .
- **JWT authentication** with short-lived (30 min) access tokens and long-lived (7 day) refresh tokens, refresh-token rotation, and a `type` claim that prevents a refresh token from being used where an access token is expected.
- **A single, consistent error envelope** — every exception, whether a domain-level `AppError` subclass, a Pydantic validation error, or an unhandled exception, is caught centrally and serialized into the same `{"error": {"code", "message", "details"}}` shape.
- **A Mongo-native migration runner** rather than Alembic — numbered migration scripts with an `up(db)` coroutine, tracked in a `migrations_history` collection, run automatically on startup. The stated reasoning in-code is that Alembic is tightly coupled to SQLAlchemy, and pulling that in purely for migrations on a Motor-based project would be unnecessary complexity.
- **Authenticated, ownership-checked media endpoints** — meal photos and avatars are streamed through `GET /meals/{id}/image` -style endpoints that verify the requester owns the resource, rather than being served as public static files.

08 — The Three-Tier Accuracy System

This is the core reliability mechanism referenced in Sections 02–05, and it's implemented essentially as originally intended, with one additional safety net beyond the three primary tiers.

`nutrition_service.resolve_nutrition()` resolves a food name to calories/macros-per-100g using this cascade:

Tier 1 — `nutrition_cache` (MongoDB collection) - *Purpose*: instant retrieval for any food already resolved once before. - *Input*: a normalized (lower-cased, trimmed) food name. - *Output*: the previously cached nutrition document, if one exists. - *Why it exists*: avoids repeat external API calls for common foods and makes repeated lookups effectively free.

Tier 2 — USDA FoodData Central API - *Purpose*: pull nutrition data from an authoritative external database when nothing is cached yet. - *Implementation*: queries USDA's `foods/search` endpoint (Foundation + SR Legacy data types), extracts energy/protein/carbohydrate/fat per 100 g from the top match. - *Output*: on success, the result is tagged `source: nutrition_api` and written into `nutrition_cache` for next time.

Tier 3 — Bundled static JSON dataset - *Purpose*: fallback for foods the USDA database doesn't have a good match for — notably useful for South Asian dishes that are under-represented in USDA data. - *Implementation*: an exact-match lookup against `app/utils/static_nutrition.json` , falling back further to a substring match if no exact key matches. - *Output*: tagged `source: static` , also cached.

Tier 4 (documented safety net, not part of the "3-tier" framing but present in code) — Conservative default - If nothing above resolves the food at all, the system returns a fixed conservative estimate (150 kcal / 5 g protein / 20 g carbs / 5 g fat per 100 g) tagged `source: ai_estimated` , and caches even that, so an unresolvable food doesn't repeatedly hit the two external tiers.

Why this matters: a single-inference pipeline (`image` → `AI` → `result`) would let a vision model's guess about *macros* stand in as ground truth. This system instead uses the AI only for what it's suited to — identifying food and estimating portion size — and resolves the actual nutrition math against cached results, an authoritative government database, or a curated dataset first, reserving the AI-estimated fallback for genuinely unmapped foods, and even then via a fixed, conservative value rather than a fresh AI guess per request. This **increases consistency** (the same food resolves to the same numbers every time once cached), **reduces dependence on a single inference** for the numeric result, and gives every stored nutrition record a visible `source` field for auditability. It is not presented, and should not be read, as medically validated or scientifically guaranteed nutrition data — it is a more structured estimation process than a bare AI guess, not a clinical-grade one.

09 — AI Engineering

Provider abstraction. `app/ai/gemini_client.py` is the only module in the codebase that imports the Gemini SDK — if

the provider ever changed, this is the single file that would need to change. A `mistral_client.py` exists with function signatures deliberately mirroring the Gemini client, but its bodies currently raise `NotImplementedError`-style errors — it is a **scaffolded, not-yet-activated** fallback provider, clearly a planned direction rather than a shipped feature.

Structured vision output. Food recognition uses Gemini's `response_mime_type="application/json"` to request a structured array directly, with a regex-based markdown-fence stripper kept as a safety net. Every item in the response is then explicitly validated — name coerced to string, quantity and confidence coerced to float with defaults — before being trusted anywhere downstream; malformed items are silently dropped rather than propagated.

Retry policy that distinguishes failure types. Rather than a blanket retry-on-any-error policy, `_is_retryable()` classifies Gemini API errors: a 429 tied to an exhausted *daily* quota is treated as non-retryable (retrying won't bring quota back before the day resets), while a 429 tied to a per-minute limit, or a 5xx/timeout, is retried with exponential backoff (2s → 4s → 8s, up to 3 attempts). Auth failures and bad requests fail immediately. This keeps the retry loop useful instead of just burning time or producing log noise on unrecoverable errors.

Centralized, swap-friendly prompts. All prompt templates live in one file (`app/ai/prompts.py`) — the food-recognition system/user prompts, the meal-summary prompt builder, and the chat system-prompt builder — so both the REST layer and the MCP server use identical prompts with no duplication.

Context-injected coaching. The chat coach's system prompt is built fresh per request from the user's actual goal, calorie target, and last five logged meals, plus recent chat history — so the coach references the user's real data rather than giving generic advice. The prompt also explicitly instructs the model to detect and match the user's language/style (English, Urdu, Hindi, or a natural Roman-Urdu/Hinglish mix) rather than defaulting to English.

10 — Backend Engineering

- **Thin routers, real services.** Every router file in the repo follows the same shape: parse the request (Pydantic does most of this automatically), delegate to a service function, return its result. No business logic lives in a router.
- **Fail-fast, fail-cheap validation.** Image uploads are checked in a strict order — MIME whitelist, then size limit, then Pillow-verified image integrity, then optional resize — *before* any Gemini call is made, so a bad upload never burns AI quota.
- **A real exception hierarchy.** `AppError` subclasses (`AuthError`, `NotFoundError`, `ForbiddenError`, `ValidationError`, `FileUploadError`, `AIServiceError`, `NutritionLookupError`, `DatabaseError`) each carry their own HTTP status and error code, all converging on one error-handling middleware that produces a single consistent JSON error shape for clients.
- **Dependency injection for cross-cutting concerns.** `CurrentUser` and `DBDep` dependency types are used consistently across routers for auth context and database access, rather than each endpoint reimplementing token parsing.
- **Auth hardening choices, with reasoning left in comments.** Password hashing uses `bcrypt` directly rather than `passlib` (the code notes `passlib`'s `bcrypt` backend breaks against `bcrypt ≥ 4.1`); JWT uses `PyJWT` rather than `python-jose` (the code cites a known `python-jose` algorithm-confusion CVE). Both are concrete, justified modernization decisions rather than default choices.

11 — Database & Data Flow

MongoDB (via Motor) with these collections, confirmed from the indexing and migration code:

Collection	Purpose	Key indexes
<code>users</code>	Account + profile (body composition, goal, calorie target)	Unique on <code>email</code>
<code>meals</code>	One document per logged meal, embedding <code>detected_foods[]</code> and <code>total_macros</code>	Compound on <code>(user_id, created_at)</code> and <code>(user_id, meal_date)</code>
<code>daily_progress</code>	One snapshot per user per calendar day, recalculated on every meal change	Unique on <code>(user_id, date)</code>
<code>chat_messages</code>	Coach conversation history, with <code>context_meal_ids</code> linking a message to the meals it was answered against	Compound on <code>(user_id, created_at)</code>
<code>nutrition_cache</code>	Resolved nutrition results from the 3-tier lookup, keyed by normalized food name	Unique on <code>food_name</code>
<code>migrations_history</code>	Tracks which schema/data migrations have run	—

The embedding decision for `detected_foods[]` and `total_macros` inside each meal document (rather than as separate referenced collections) is a deliberate choice noted directly in the model file: these fields are always read together with the meal, so embedding avoids unnecessary joins for the access pattern that actually matters.

12-13 — Key Engineering Challenges & Decisions

1. Trusting AI for perception, not for numbers. *Challenge:* a single `image → AI → result` pipeline makes the whole product's credibility depend on one ungrounded inference per meal. *Decision:* split the pipeline — AI identifies and estimates portions; a separate, cacheable, multi-source lookup resolves the actual calorie/macro numbers. *Result:* numeric output is grounded in cached results or an authoritative external database wherever possible (Section 08), with AI-estimated values used only as a last, clearly-labeled resort.

2. Handling Gemini failures without wasting retries or hiding real errors. *Challenge:* not all API failures deserve the same response — an exhausted daily quota, a per-minute rate limit, and a genuine 5xx all look similar from the outside but call for different handling. *Decision:* classify errors before deciding whether to back off. *Implementation:* `_is_retryable()` inspects the error's status/message to separate daily-quota exhaustion (fail fast) from per-minute limits and transient 5xx/timeouts (retry with exponential backoff). *Result:* the coach surfaces a friendly "temporarily unavailable" message instead of a raw stack trace, and only retries failures actually worth retrying.

3. Keeping calorie targets correct without duplicating math on the frontend. *Challenge:* the Mifflin-St Jeor formula shouldn't need to be reimplemented client-side, but a naive "recalculate whenever the profile endpoint is hit" approach would misfire — the edit-profile screen always resends the current `daily_calorie_target` for display, so its mere presence in a `PATCH` isn't a reliable signal that anything changed. *Decision:* diff the *specific* fields the formula depends on (weight, height, age, gender, activity level, goal) against the stored document before recomputing, and only recompute — and refresh today's progress snapshot — when one of those actually changed value. *Result:* the calorie target and dashboard stay accurate automatically without the risk of silently overwriting a user's manual override.

4. Not exposing user media as public static files. *Challenge:* meal photos and avatars are personal; serving them from a public `/uploads/` path would let anyone with a guessed or leaked filename view another user's food photos. *Decision:* serve all media exclusively through authenticated endpoints that check resource ownership before streaming the file. *Result:* private-by-default media, at the cost of a slightly heavier request path than static hosting — a reasonable trade for a personal-data product.

5. Schema evolution on MongoDB without adopting a SQL-oriented migration framework. *Challenge:* the project needed versioned, repeatable migrations, but Alembic assumes SQLAlchemy. *Decision:* build a small, purpose-built Mongo-native migration runner instead. *Result:* numbered migration scripts, tracked in their own collection, applied automatically at startup — the same guarantee Alembic gives, without an unrelated ORM dependency.

14 — What I Learned

- Designing an AI feature well often means deciding what the AI is *not* allowed to be the source of truth for — letting Gemini handle perception while keeping calorie math deterministic was the single highest-leverage architectural decision in this project.
 - Defensive parsing is not optional with multimodal AI output — every field coming back from a vision call needs an explicit validation/coercion step before it's trusted, no matter how reliable the model usually is.
 - Retry logic needs to be error-aware. A blanket retry policy either wastes time retrying a dead daily quota, or worse, silently retries something that should have failed loudly.
 - Exposing the exact same service layer through both a REST API and an MCP server (rather than writing a second, parallel implementation for agentic clients) keeps business logic in one place and removes an entire class of "the two surfaces disagree" bugs.
 - Translating a fairly abstract product question ("can I make AI-assisted nutrition tracking more reliable?") into concrete choices — a caching layer, an external data source, a fallback dataset, and a documented last-resort default — was the real design work; the AI call itself was the easy part.
-

15 — Future Improvements *(not yet implemented — planned directions only)*

- **Activate the Mistral fallback provider.** The client module is scaffolded with a Gemini-matching interface but currently raises "not yet activated" errors — wiring it in would give the AI layer a second-provider fallback.
 - **Expand the static nutrition dataset** for broader regional/South Asian food coverage, reducing how often Tier 3/4 of the lookup cascade is needed.
 - **Broaden automated test coverage.** The current suite covers password hashing, JWT issuance, TDEE calculation, prompt-string construction, and one integration flow (register → login) — solid but narrow; meal-scanning, nutrition resolution, and MCP tool paths aren't yet covered.
 - **Stronger portion-size validation** — e.g. cross-checking Gemini's gram estimates against typical serving-size ranges per food.
 - **Production-scale deployment tooling** beyond the current Docker + docker-compose (app + MongoDB) setup — no evidence in the repository of a deployed environment, orchestration layer, or monitoring stack beyond that.
-

16 — Project Impact *(qualitative — no metrics are fabricated)*

- Demonstrates an independently built, production-shaped backend combining multimodal AI, a deterministic data-resolution layer, and an agentic (MCP) integration surface.
 - Replaces a naive single-AI-call nutrition pipeline with a cached, multi-source resolution strategy, directly addressing the reliability question the project set out to answer.
 - Provides one shared service layer consumed by two different client types — a conventional REST API and an MCP tool surface for autonomous agents — without duplicating business logic.
 - Establishes a consistent architectural pattern (strict layering, centralized prompts, one error envelope, a custom Mongo-native migration system) applied uniformly across every domain in the codebase.
-

17 — Technology Stack

Backend Python 3.12+, FastAPI, Pydantic v2 + pydantic-settings, Uvicorn

Database MongoDB 7 via Motor (async driver) + PyMongo, custom Mongo-native migration runner

AI Google Gemini — [google-genai](#) SDK (Vision + Chat); USDA FoodData Central API for nutrition lookups; Mistral client

scaffolded (not yet active)

Auth & Security PyJWT, bcrypt, MIME-type whitelisting + Pillow image-integrity validation, ownership-checked media streaming

Agentic Integration Model Context Protocol (MCP) server exposing `log_meal`, `get_daily_progress`, `get_weekly_progress`, `chat_with_coach`, and `lookup_nutrition` as tools for MCP-compatible clients (e.g. Claude Desktop)

Dev / Infrastructure Docker + docker-compose (app + MongoDB 7), pytest + pytest-asyncio

18 — My Role

Built independently as an **Agentic AI Engineer / Python Backend Engineer** — architecture, all backend code, the AI integration layer (prompts, retry logic, provider abstraction), the nutrition-resolution system, and the MCP server were designed and implemented solo. The work spans the full backend surface: API design, database schema and indexing, authentication and security hardening, the multi-tier nutrition-reliability system, and exposing the same business logic to both human-facing (REST) and agentic (MCP) clients. Neutria AI also has a companion React Native/Expo mobile client consuming this backend, built as part of the same broader project.

B. Portfolio Version *(condensed)*

Neutria AI — AI-Powered Nutrition Coach Backend

Neutria AI is a FastAPI + MongoDB backend for an AI-assisted nutrition-tracking product. A user photographs a meal; Gemini Vision identifies the food items and estimates portion sizes; but instead of trusting the AI for the calorie and macro numbers too, those are resolved through a **3-tier deterministic lookup** — a MongoDB cache, the USDA FoodData Central API, and a bundled static dataset (with a clearly-labeled conservative fallback for the rare unmapped food). Every resolved value is cached and tagged with its source for auditability.

The backend also powers a context-aware AI coaching chat — grounded in the user's real profile, goal, and recent meals, and able to match the user's own language (English, Urdu, Hindi, or Hinglish) — and exposes its core capabilities as **Model Context Protocol (MCP) tools**, so autonomous AI clients like Claude Desktop can log meals or check progress directly, using the exact same service layer as the REST API.

Architecturally, it follows a strict Router → Service → Repository separation, a single consistent JSON error envelope, JWT auth with refresh-token rotation, fail-fast upload validation before any AI call, and a custom Mongo-native migration runner. Built independently, end to end.

Stack: Python · FastAPI · MongoDB (Motor) · Google Gemini · USDA FoodData Central · JWT · MCP · Docker

C. Project Summary *(for a project card)*

Neutria AI is an independently built, AI-powered nutrition-coaching backend that turns a meal photo into a calorie and macro breakdown without trusting a single AI call for the numbers that matter. Gemini Vision identifies food and portions, while a 3-tier deterministic lookup — cache, USDA FoodData Central, and a curated static dataset — resolves the actual nutrition data. It also exposes its core features as MCP tools for autonomous AI clients, built on a FastAPI + MongoDB backend with strict layered architecture, JWT auth, and a context-aware coaching chat.

D. Key Highlights

- Multimodal food recognition via Gemini Vision, with structured JSON output and defensive validation of every field

- 3-tier deterministic nutrition-lookup system (cache → USDA FoodData Central → static dataset) that keeps calorie/macro numbers out of the AI's hands
- Context-aware AI coaching chat that references the user's real meal history and goal, and matches their language/style
- MCP server exposing 5 tools (`log_meal` , `get_daily_progress` , `get_weekly_progress` , `chat_with_coach` , `lookup_nutrition`) backed by the same service layer as the REST API
- Strict Router → Service → Repository architecture with a single consistent JSON error envelope
- JWT auth with refresh-token rotation, bcrypt hashing, and fail-fast upload validation before any AI call
- Custom Mongo-native migration runner — versioned schema changes without pulling in SQLAlchemy/Alembic
- Self-recalculating TDEE-based calorie target (Mifflin-St Jeor) that updates automatically only when a body-composition field genuinely changes

E. Case Study Metadata

Field	Value
Project title	Neutria AI
Tagline	An AI nutrition coach that doesn't let AI guess your calories.
Role	Agentic AI Engineer / Python Backend Engineer (independent)
Category	AI-powered backend system / Agentic AI
Tech stack	Python, FastAPI, MongoDB (Motor), Google Gemini, USDA FoodData Central, JWT, MCP, Docker
Key capabilities	Multimodal food recognition, deterministic 3-tier nutrition resolution, context-aware AI coaching, MCP tool exposure
Repository	https://github.com/abizaid24/Neutria-AI-Backend-
Live demo	<i>(not available / add link)</i>